

Adversarial Search : Minimax

Algorithm for two player games

Introduction

In the realm of Artificial Intelligence, search algorithms are fundamental, often guiding an agent through a problem space to a desired goal. Traditionally, these algorithms, such as depth-first or breadth-first search, operate under the assumption of a single, atomic agent with complete control over its actions and predictable outcomes. However, the real world, and indeed many compelling AI challenges, involve multiple agents whose goals may conflict. This lecture introduces the concept of adversarial search, a critical paradigm shift from single-agent problems, focusing specifically on two-player games.

Game playing has long been a cornerstone for demonstrating artificial intelligence, offering a structured yet complex environment for AI research. The appeal lies in their formal nature: rules are explicit, states are well-defined, and outcomes are clear. Unlike a robot navigating a chaotic physical environment, games provide a controlled setting where the core challenge is reasoning and strategy. Furthermore, games allow for direct, measurable comparisons between AI systems and human performance, providing a tangible benchmark for intelligence. This lecture will delve into how AI approaches these competitive scenarios, laying the groundwork for understanding algorithms like Minimax that enable machines to play strategically against intelligent opponents.

Core Concepts

Term	Definition	Significance
Adversarial Search	A type of search problem where multiple agents, with conflicting goals, interact within the same environment, requiring an agent to consider the actions of its opponents.	Essential for modeling competitive scenarios, moving beyond single-agent problem-solving to strategic interaction.
Single-Agent Search	Search algorithms where a single agent operates in an environment, and all actions	The foundational search paradigm from which adversarial search diverges, highlighting the unique

Term	Definition	Significance
	lead to predictable outcomes controlled solely by that agent.	challenges of multi-agent systems.
Two-Player Games	Games involving two agents, typically with opposing objectives, where one agent's gain is often the other's loss.	The primary focus of this lecture, providing a simplified yet rich context for understanding adversarial decision-making.
Optimal Adversaries	The assumption that an opponent will always make the best possible move for themselves, which often translates to the worst possible outcome for the current player.	A crucial assumption in Minimax, ensuring the algorithm plans for the most challenging opponent, leading to robust strategies.
Utility Function	A numerical value assigned to terminal states of a game, representing the desirability of that state for a particular player (e.g., +1 for win, 0 for draw, -1 for loss).	Quantifies the outcome of a game, allowing the search algorithm to evaluate and compare different game-ending scenarios.
Max Node	A node in the search tree representing a turn where the current player (the AI) makes a move, aiming to maximize its utility.	Represents the AI's decision points, where it seeks to achieve the highest possible score.
Min Node	A node in the search tree representing a turn where the opponent makes a move, aiming to minimize the current player's utility.	Represents the opponent's decision points, where they seek to inflict the lowest possible score on the AI.
Minimax Search Tree	A search tree where nodes alternate between Max nodes (player's turn) and Min nodes (opponent's turn), used to	The core data structure for the Minimax algorithm, visualizing the game's

Term	Definition	Significance
	determine the optimal move for a player.	possible futures and strategic choices.
Ply	A single action taken by one player in a game. A "move" often refers to a pair of plies (one by each player).	Clarifies the granular steps within a game, distinguishing individual actions from a complete turn cycle.
Backing Up Values	The process of propagating utility values from the terminal (leaf) nodes of a search tree upwards to intermediate nodes, based on whether the node is a Max or Min node.	Enables the evaluation of non-terminal states by inferring their long-term outcome, crucial for making current strategic decisions.

Detailed Analysis

1. Characteristics of Games and the Shift to Adversarial Search

Games, particularly two-player sequential games, introduce fundamental differences compared to the single-agent search problems previously discussed. In a game, players make moves, and an adversary responds. While some games might involve simultaneous moves (like Rock-Paper-Scissors), the focus here is on sequential turns, where one player acts, then the other. Each move is governed by specific rules, defining what actions are legal (e.g., pawn movement in chess). Rewards, or utility, are often concentrated at the end of the game (win, lose, draw), though some games might offer intermediate rewards.

The critical distinction from single-agent search lies in the loss of control. In single-agent problems, an action leads to a known next state, and the sequence of steps is entirely within the agent's control. In adversarial games, after a player makes a move, control is relinquished to the opponent. The next state is not solely determined by the player's action but also by the opponent's response, which is designed to counter the player's objectives. This necessitates a different approach to search, where the AI must anticipate and prepare for various opponent actions, rather than simply planning a linear sequence of its own moves.

Furthermore, real-world games often have time limits, making it impractical to search to the absolute end of every possible game path. Games like chess can involve hundreds of moves, leading to an astronomically large search space. Therefore, adversarial search algorithms frequently rely on approximation,

evaluating states at a certain depth rather than exhaustively searching to terminal states. The general workflow for a two-player sequential game involves:

- An opponent makes a move, creating a new board position.
- The system checks if the opponent has won or lost.
- If not, the system generates all possible successor states from the current board position.
- These successors are evaluated.
- The system selects the action leading to the "best" successor.
- The process repeats.

2. Components of a Search Algorithm in Games

To apply search algorithms to games, several components must be defined, similar to single-agent search but adapted for the adversarial context:

- **State:** This refers to the current configuration of the game, such as the arrangement of pieces on a chessboard.
- **Initial State:** The starting configuration of the game.
- **Successor Function:** This function determines all legal moves available from a given state for the current player (either the AI or the adversary) and the resulting states after each move.
- **Terminal Test:** A condition that determines if the game has ended. This includes checking for a win, a loss, or a draw.
- **Utility Function:** A numerical value assigned to terminal states. For simplicity, this is often defined as +1 for a win, 0 for a draw, and -1 for a loss, from the perspective of the AI.

3. The Adversarial Nature: Max vs. Min

The core of adversarial search lies in the conflicting goals of the players. If the AI (the computer) aims to maximize its utility (e.g., achieve a +1 win), its opponent (e.g., a human player) will simultaneously try to maximize their utility. From the computer's perspective, the opponent's goal of maximizing their own utility is equivalent to minimizing the computer's utility. This creates a "zero-sum" game dynamic where one player's gain is the other's loss.

This fundamental opposition leads to the concept of "Max" and "Min" nodes in the search tree:

- **Max Nodes:** Represent turns where the AI is making a move. At these nodes, the AI will choose the action that leads to the maximum possible utility for itself, assuming optimal play from the opponent thereafter.
- **Min Nodes:** Represent turns where the opponent is making a move. At these nodes, the AI assumes the opponent will choose the action that leads to the

minimum possible utility for the AI, again assuming optimal play from the AI thereafter.

The search tree thus alternates between Max and Min nodes, reflecting the turn-taking nature of the game and the opposing objectives of the players.

4. Minimax Search Tree and Backing Up Values

The search tree generated for adversarial games is known as the Minimax search tree. It visualizes all possible sequences of moves from the current state to a terminal state. The process involves:

1. **Building the Tree:** Starting from the current game state, generate all possible moves for the current player, then all possible responses for the opponent, and so on, to a certain depth or until terminal states are reached.

2. **Applying Utility Function at Leaves:** Once terminal states are reached (the leaves of the tree), the utility function is applied to assign a numerical value (+1, 0, or -1) to each, representing the outcome for the AI.

3. **Backing Up Values:** The crucial step is to propagate these utility values up the tree to the intermediate nodes. This is done by assuming optimal play at each step:

- **At Min Nodes (Opponent's Turn):** The value of a Min node is the minimum of the utility values of its children. The opponent will choose the move that results in the lowest utility for the AI.

- **At Max Nodes (AI's Turn):** The value of a Max node is the maximum of the utility values of its children. The AI will choose the move that results in the highest utility for itself.

Consider a simple 2-step game example:

- The AI (Max) is at the root. It can choose actions A1, A2, or A3.
- Below each of these, the opponent (Min) can choose from several actions (e.g., A11, A12, A13 for A1).
- At the leaves, the final utility values for the AI are given (e.g., 3, 12, 8 for A11, A12, A13 respectively).

To determine the optimal move for the AI:

- If the AI chooses A1, the opponent will then choose between A11 (utility 3), A12 (utility 12), and A13 (utility 8). Since the opponent is a Min player, they will choose A11, resulting in a utility of 3 for the AI. So, the backed-up value for A1 is 3.
- Similarly, if the AI chooses A2, the opponent might choose a move leading to a utility of 2. So, the backed-up value for A2 is 2.
- If the AI chooses A3, the opponent might choose a move leading to a utility of 2. So, the backed-up value for A3 is 2.

Finally, at the root (Max node), the AI compares the backed-up values for A1 (3), A2 (2), and A3 (2). As a Max player, the AI will choose the action that maximizes its

utility, which is A1, leading to a guaranteed utility of 3. This process not only identifies the best initial move but also anticipates the opponent's optimal responses throughout the game.

Key Takeaways

- [] Adversarial search differs from single-agent search by involving multiple agents with conflicting goals.
- [] Game playing is a valuable domain for AI research due to its formal rules, clear objectives, and direct human-AI comparison.
- [] In adversarial games, the AI loses control after its move, as the opponent's actions are designed to minimize the AI's utility.
- [] The Minimax algorithm assumes optimal play from both the AI (maximizing its utility) and the opponent (minimizing the AI's utility).
- [] A Minimax search tree alternates between Max nodes (AI's turn) and Min nodes (opponent's turn).
- [] Utility values are assigned to terminal game states and then "backed up" the tree, with Max nodes taking the maximum child value and Min nodes taking the minimum child value.
- [] The optimal move for the AI is the one that leads to the highest backed-up utility value at the root of the Minimax tree.
- [] Due to time constraints, adversarial search often involves approximating game outcomes rather than searching to the absolute end of every game path.

Conclusion

The Minimax algorithm provides a foundational framework for strategic decision-making in two-player, zero-sum games. By systematically exploring possible game states and anticipating an optimal adversary's moves, AI systems can determine the best course of action to maximize their chances of winning. This approach moves beyond simple goal-seeking to embrace the complexities of interaction with intelligent opponents, a crucial step towards more sophisticated AI.

While the basic Minimax algorithm can be computationally intensive for complex games, its core principles of maximizing one's own utility while minimizing the opponent's form the basis for more advanced adversarial search techniques. Understanding Minimax is essential for appreciating how AI can navigate competitive environments, from classic board games to more intricate real-world scenarios where multiple agents vie for success. This strategic thinking is a

cornerstone of developing AI that can not only solve problems but also compete and adapt in dynamic, multi-agent settings.